Claude Code Reference — Atram Engineering

Built-in CLI commands and **compound-engineering** plugin skills (by Kieran Klaassen / Every), grouped by what you'd reach for them to do. Updated 2026-05-22.

Autonomous loops — set a target, walk away

For work that benefits from multiple turns without supervision: tests that flake, lints to clean, plans to draft, reviews to gather. Always bound them with a turn cap or a clear exit condition.

`/goal` built-in

Sets a completion condition; Claude loops turn-after-turn while a Haiku evaluator checks the transcript. Stops when the condition holds or you `/goal clear`. Survives `--resume`.

> **Use it for:** "`pytest tests/alerts runs 10× green, or stop after 25 turns`" — flaky-test hunts, lint-clean before a PR, migration backfill parity checks.

`/ultraplan` built-in

Offloads planning to a cloud session. Draft a multi-step plan collaboratively in the browser, annotate sections, then execute locally or in cloud.

> **Use it for:** the kickoff of a non-trivial Atram feature — e.g. designing the ET WhatsApp/Telegram channel split before writing code.

`/ultrareview` built-in

Spawns a multi-agent cloud review of the current branch (or a GitHub PR). Each reviewer covers a different angle (security, perf, style) and aggregates findings.

> **Use it for:** a final sweep before merging yaya-engine work to `dev`. Run `/ultrareview <PR#>` from the terminal — billed, user-triggered.

`/slfg` CE

Compound-engineering's *full autonomous engineering workflow*: plan → implement → review, all in swarm mode with parallel sub-agents. The biggest gun in the kit.

> **Use it for:** end-to-end feature work where you're happy to come back to a finished PR (and don't mind paying for the agents). Pair with a tight ticket description.

`/compound-engineering:orchestrating-swarms` CE

Lower-level than `/slfg` — packages Claude Code's `Task` tool + TeammateTool messaging into a swarm protocol with file-backed JSON state, dependency ordering, leader/worker roles, inbox-style coordination.

> **Use it for:** bespoke multi-agent jobs you want reproducible across runs — e.g. one agent per repo scanning for hardcoded secrets, one agent per language translating WhatsApp templates with placeholder validation, parallel forecast-variant evals against a shared eval set.

Session control — manage the conversation

Built-in commands for navigating long sessions, recovering from dead-ends, moving between local/cloud, and changing how you talk to Claude.

`/recap` built-in

Summarises what happened in the session while you were away. Useful after a long autonomous run or a multi-day resumed session.

> **Use it for:** waking up to a `/goal` that finished overnight — get the gist without scrolling through 40 turns of tool calls.

`/rewind` built-in

Rolls code *and* conversation back to a checkpoint. Also bound to `Esc Esc`. Lets you explore a risky approach and undo cleanly.

> **Use it for:** trying a speculative refactor in yaya-engine without polluting your branch — rewind if it doesn't pan out.

`/teleport` built-in

Moves a session between your terminal and the web (and back). Full conversation history travels with it.

> **Use it for:** kicking off a long task in the cloud from your laptop, then teleporting it locally when you need filesystem access or your editor.

`/voice` built-in

Toggles speech-to-text dictation for prompts. Modes: `/voice hold` (push-to-talk), `/voice tap` (toggle).

> **Use it for:** long context dumps (describing a bug, walking through a feature) when typing slows you down. Caveat: punctuation needs cleanup.

`/radio` built-in

Opens Claude FM (music streaming) in your browser. Pure background-noise utility, no engineering purpose.

> **Use it for:** a soundtrack while a long swarm or `/goal` chugs in another pane. That's it.

Authoring & extending Claude Code

Tools for shaping how Claude Code behaves on this project — bootstrapping config, writing reusable skills, designing systems where agents are first-class.

`/setup` CE

Diagnoses the project's dev environment, installs missing tools, and bootstraps the compound-engineering config (CLAUDE.md, hooks, allowlists). One-shot project init.

> **Use it for:** onboarding a new Atram repo to compound-engineering — point it at `yaya-frontend` or `atram-utils-webapp` and let it scaffold.

`/skill-creator` CE

Guided authoring of new Claude Code skills — generates a SKILL.md, frontmatter, triggers, and the recommended file layout. Includes review of existing skill patterns.

> **Use it for:** turning a recurring Atram workflow (e.g. "create alert template + push to WABA + verify webhook") into a reusable `/atram-alert-publish` skill.

`/agent-native-architecture` CE

Design guidance for building systems where agents are first-class citizens — MCP tools, self-modifying loops, applications whose features are outcomes that agents achieve. Pushes you to think tool-first, not UI-first.

> **Use it for:** early design of new internal tooling (alert-manager rewrite, ETL pipelines) where the operators will be a mix of humans and agents. Output: a list of MCP tools to build, with their contracts.

Testing

Skills that drive real test runs — browser pages, iOS simulators — so the agent can see results, not just imagine them.

`/test-browser` CE

Runs browser tests on pages affected by the current PR or branch. Spins up a headless browser, exercises flows, returns screenshots and console errors.

> **Use it for:** verifying the atram-utils-webapp geocoder widget after a CSS or JS change — confirm the address search still resolves and the map renders.

`/test-xcode` CE

Builds and runs iOS apps on the simulator, captures test results, and feeds failures back to the agent for fixing.

> **Use it for:** a-pachas-ios changes — run the test suite on a simulator without leaving the terminal, let Claude iterate on failing assertions.

Data, files & frameworks

Skills for moving files between clouds, telling stories with data, and building production LLM features in Ruby.

`/rclone` CE

Uploads / syncs files across cloud providers via rclone — S3, R2, B2, GDrive, Dropbox. Handles auth, retries, and large transfers.

> **Use it for:** shipping forecast outputs from `weather-model-evaluations` to S3, syncing template assets to `pdf-hosting-atram`, backing up large JSON dumps.

`/data-storytelling` CE

Transforms raw analytics into stakeholder-ready narratives — visualisation choice, context, persuasive structure. Outputs charts, decks, executive summaries.

> **Use it for:** turning a Colombia user-engagement query result into a one-slide narrative for an ops review, or framing alert delivery stats for a partner report.

`/dspy-ruby` CE

Build type-safe LLM features in Ruby using DSPy.rb — programmatic prompts via signatures and modules, agent systems with tools, prompt optimisation, and tests. The Ruby answer to building predictable, testable AI features.

> **Use it for:** any Rails-side AI feature (alert summarisation, message classification, user-intent extraction) where you want the LLM call to look like a typed method, not a fragile prompt string.

Sources: Built-in commands: `code.claude.com/docs/en/` · Compound-engineering plugin: `github.com/EveryInc/compound-engineering-plugin` · Every guide: `every.to/guides/compound-engineering` · Swarms reference: `gist.github.com/kieranklaassen/4f2aba89594a4aea4ad64d753984b2ea` · Source file: `Atram-Inc/.github/docs/claude-code/goal-and-swarms.html`